

# Tecnología Digital VI

## Ensamblés II – Camino e Interpretabilidad

Licenciatura en Tecnología Digital – UTDT

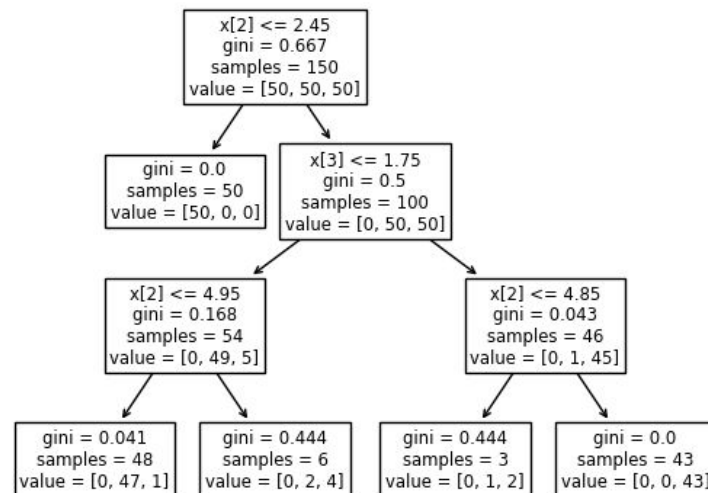
# Interpretación

Los árboles de decisión son *muy* interpretables. Existen librerías que pueden dibujarnos el árbol completo ([https://scikit-learn.org/stable/modules/generated/sklearn.tree.plot\\_tree.html](https://scikit-learn.org/stable/modules/generated/sklearn.tree.plot_tree.html)).

Los Random Forest son mucho más complejos, ¿serán una caja negra?

```
from sklearn.datasets import load_iris
from sklearn import tree
clf = tree.DecisionTreeClassifier(random_state=42, max_depth=3)
iris = load_iris()
clf = clf.fit(iris.data, iris.target)
tree.plot_tree(clf)
```

```
[Text(0.375, 0.875, 'x[2] <= 2.45\ngini = 0.667\nsamples = 150\nvalue = [50, 50, 50]'),
Text(0.25, 0.625, 'gini = 0.0\nsamples = 50\nvalue = [50, 0, 0]'),
Text(0.5, 0.625, 'x[3] <= 1.75\ngini = 0.5\nsamples = 100\nvalue = [0, 50, 50]'),
Text(0.25, 0.375, 'x[2] <= 4.95\ngini = 0.168\nsamples = 54\nvalue = [0, 49, 5]'),
Text(0.125, 0.125, 'gini = 0.041\nsamples = 48\nvalue = [0, 47, 1]'),
Text(0.375, 0.125, 'gini = 0.444\nsamples = 6\nvalue = [0, 2, 4]'),
Text(0.75, 0.375, 'x[2] <= 4.85\ngini = 0.043\nsamples = 46\nvalue = [0, 1, 45]'),
Text(0.625, 0.125, 'gini = 0.444\nsamples = 3\nvalue = [0, 1, 2]'),
Text(0.875, 0.125, 'gini = 0.0\nsamples = 43\nvalue = [0, 0, 43]')]
```



# Bulldozers

Los dataset que venimos utilizando en esta presentación tienen pocas columnas. Vamos a utilizar un dataset más complejo con mucha más información.

El estudio de este dataset lo pueden encontrar en el capítulo 9 del libro de Jeremy Howard.

 Featured Prediction Competition

## Blue Book for Bulldozers

Predict the auction sale price for a piece of heavy equipment to create a "blue book" for bulldozers.

474 teams · 10 years ago

Overview

Data

Code

Discussion

Leaderboard

Rules

Team

...

\$10,000

Prize Money

## Dataset Description

### [View and download the benchmark code from Github](#)

For this competition, you are **predicting the sale price of bulldozers sold at auctions**.

The data for this competition is split into three parts:

- **Train.csv** is the training set, which contains data through the end of 2011.
- **Valid.csv** is the validation set, which contains data from January 1, 2012 - April 30, 2012 You make predictions on this set throughout the majority of the competition. Your score on this set is used to create the public leaderboard.
- **Test.csv** is the test set, which won't be released until the last week of the competition. It contains data from May 1, 2012 - November 2012. Your score on the test set determines your final rank for the competition.

The key fields are in train.csv are:

- SalesID: the unique identifier of the sale
- MachineID: the unique identifier of a machine. A machine can be sold multiple times
- saleprice: what the machine sold for at auction (only provided in train.csv)
- saledate: the date of the sale

There are several fields towards the end of the file on the different options a machine can have. The descriptions all start with "machine configuration" in the data dictionary. Some product types do not have a particular option, so all the records for that option variable will be null for that product type. Also, some sources do not provide good option and/or hours data.

The machine\_appendix.csv file contains the correct year manufactured for a given machine along with the make, model, and product class details. There is one machine id for every machine in all the competition datasets (training, evaluation, etc.).

### Files

14 files

### Size

213.81 MB

### Type

csv, zip, 7z

# Bulldozers – Data

Tenemos un dataset donde el objetivo es predecir el precio de venta a partir de muchas columnas que describen maquinaria de construcción.

```
df = pd.read_csv('./bluebook-for-bulldozers/TrainAndValid.csv')
```

```
df.columns
```

```
Index(['SalesID', 'SalePrice', 'MachineID', 'ModelID', 'datasource',  
      'auctioneerID', 'YearMade', 'MachineHoursCurrentMeter', 'UsageBand',  
      'saledate', 'fiModelDesc', 'fiBaseModel', 'fiSecondaryDesc',  
      'fiModelSeries', 'fiModelDescriptor', 'ProductSize',  
      'fiProductClassDesc', 'state', 'ProductGroup', 'ProductGroupDesc',  
      'Drive_System', 'Enclosure', 'Forks', 'Pad_Type', 'Ride_Control',  
      'Stick', 'Transmission', 'Turbocharged', 'Blade_Extension',  
      'Blade_Width', 'Enclosure_Type', 'Engine_Horsepower', 'Hydraulics',  
      'Pushblock', 'Ripper', 'Scarifier', 'Tip_Control', 'Tire_Size',  
      'Coupler', 'Coupler_System', 'Grouser_Tracks', 'Hydraulics_Flow',  
      'Track_Type', 'Undercarriage_Pad_Width', 'Stick_Length', 'Thumb',  
      'Pattern_Changer', 'Grouser_Type', 'Backhoe_Mounting', 'Blade_Type',  
      'Travel_Controls', 'Differential_Type', 'Steering_Controls'],  
      dtype='str')
```

# Bulldozers – Preprocesamiento

Los datos tienen muchas variables numéricas pero también hay otros tipos de variables y datos faltantes, lo cual sería un problema para usar un Random Forest de Scikit-Learn.

Comenzamos por preprocesar una variable categórica ordinal como el tamaño del producto.

```
df['ProductSize'].unique()
```

```
<ArrowStringArray>  
[nan, 'Medium', 'Small', 'Large / Medium', 'Mini', 'Large', 'Compact']  
Length: 7, dtype: str
```

```
sizes = 'Large', 'Large / Medium', 'Medium', 'Small', 'Mini', 'Compact'
```

```
df['ProductSize'] = df['ProductSize'].astype('category')  
df['ProductSize'] = df['ProductSize'].cat.set_categories(sizes, ordered=True)
```

```
df['ProductSize'].head()
```

```
0      NaN  
1    Medium  
2      NaN  
3     Small  
4      NaN
```

```
Name: ProductSize, dtype: category
```

```
Categories (6, str): ['Large' < 'Large / Medium' < 'Medium' < 'Small' < 'Mini' < 'Compact']
```

# Bulldozers – Métrica

La variable dependiente en este caso es SalePrice, el precio de venta. En la competencia original la métrica a optimizar era el RMSLE en vez del RMSE (¿Por qué?).

Para optimizar esta métrica, simplemente vamos a tomar el logaritmo y luego seguir con RMSE.

```
dep_var = 'SalePrice'  
df[dep_var] = np.log(df[dep_var])
```

# Bulldozers – Fechas

El dataframe tiene una columna de tipo “datetime” (una fecha) que no solo no es numérica sino que además tiene un montón de información con la que podemos hacer feature engineering.

Como preprocesamiento, creamos un montón de columnas nuevas a partir de la fecha.

```
df['saledate'] = pd.to_datetime(df['saledate'])  
df = add_datepart(df, 'saledate')
```

```
df_test = pd.read_csv('./bluebook-for-bulldozers/Test.csv')  
df_test['saledate'] = pd.to_datetime(df_test['saledate'])  
df_test = add_datepart(df_test, 'saledate')
```

```
print('\n'.join(column for column in df.columns if column.startswith('sale')))
```

```
saleYear  
saleMonth  
saleWeek  
saleDay  
saleDayofweek  
saleDayofyear  
saleIs_month_end  
saleIs_month_start  
saleIs_quarter_end  
saleIs_quarter_start  
saleIs_year_end  
saleIs_year_start  
saleElapsed
```

# Bulldozers – Preprocesamiento

Al analizar las columnas de nuestro dataframe vemos que hay una llamada YearMade correspondiente al año de fabricación de la maquinaria. Muchas filas tienen un valor 1000 en esta columna.

Con casi 100% de seguridad esas máquinas no se fabricaron en el año 1000 si no que es un placeholder para decir que no sabemos el valor.

```
df['YearMade'] = df.YearMade.replace(1000, np.nan)  
df_test['YearMade'] = df_test.YearMade.replace(1000, np.nan)
```

# Bulldozers – Preprocesamiento

Para utilizar un Random Forest de Scikit-learn tenemos dos grandes problemas: los valores no numéricos y los valores faltantes.

En este ejemplo vamos a usar la librería `fastai` que puede preprocesar dataframes con diferentes transformaciones. En particular vamos a usar `Categorify` y `FillMissing` para los dos problemas mencionados.

```
procs = [Categorify, FillMissing]
```

# Bulldozers – Train y Validation

En este ejemplo *no podemos* utilizar la funcionalidad `train_test_split` de `sklearn` que veníamos usando. ¿Por qué?

Vamos a crear nuestros conjuntos de entrenamiento y validación a mano partiendo en una fecha en particular.

```
cond = (df.saleYear<2011) | (df.saleMonth<10)
train_idx = np.where( cond)[0]
valid_idx = np.where(~cond)[0]

splits = (list(train_idx),list(valid_idx))
```

# Bulldozers – Preprocesamiento

Lo último que necesitamos para utilizar la librería fastai es usar la funcionalidad `cont_cat_split` para separar las variables categóricas de las continuas y así poder realizar el preprocesamiento.

Utilizando TabularPandas vamos a conseguir un objeto que tiene toda la información necesaria preprocesada.

```
cont,cat = cont_cat_split(df, 1, dep_var=dep_var)
```

Una vez definidos los preprocesamientos, se utiliza la clase TabularPandas con toda esta información.

```
to = TabularPandas(df, procs, cat, cont, y_names=dep_var, splits=splits)
```

```
len(to.train),len(to.valid)
```

```
(404710, 7988)
```

# Bulldozers – Preprocesamiento

Una vez hecho el preprocesamiento, se puede ver que ya no tenemos datos que no sean numéricos (ni tampoco datos faltantes).

```
to.show(3)
```

|   | UsageBand | fiModelDesc | fiBaseModel | fiSecondaryDesc | fiModelSeries | fiModelDescriptor | ProductSize | fiProductClassDesc                                         |
|---|-----------|-------------|-------------|-----------------|---------------|-------------------|-------------|------------------------------------------------------------|
| 0 | Low       | 521D        | 521         | D               | #na#          | #na#              | #na#        | Wheel Loader - 110.0 to 120.0 Horsepower                   |
| 1 | Low       | 950FII      | 950         | F               | II            | #na#              | Medium      | Wheel Loader - 150.0 to 175.0 Horsepower                   |
| 2 | High      | 226         | 226         | #na#            | #na#          | #na#              | #na#        | Skid Steer Loader - 1351.0 to 1601.0 Lb Operating Capacity |



```
to.items.head(3)
```

|   | SalesID | SalePrice | MachineID | ModelID | datasource | auctioneerID | YearMade | MachineHoursCurrentMeter | UsageBand | fiM |
|---|---------|-----------|-----------|---------|------------|--------------|----------|--------------------------|-----------|-----|
| 0 | 1139246 | 11.097410 | 999089    | 3157    | 121        | 3.0          | 2004.0   | 68.0                     | 2         |     |
| 1 | 1139248 | 10.950807 | 117657    | 77      | 121        | 3.0          | 1996.0   | 4640.0                   | 2         |     |
| 2 | 1139249 | 9.210340  | 434808    | 7009    | 121        | 3.0          | 2001.0   | 2838.0                   | 1         |     |

# Bulldozers – Preprocesamiento

Utilizando el método *info()* de pandas vemos que ahora todas las columnas son de tipo numérico y que no hay *nulls*.

```
to.train.xs.info()
```

```
<class 'pandas.DataFrame'>  
Index: 404710 entries, 0 to 412697
```

```
Data columns (total 67 columns):
```

| #  | Column             | Non-Null Count  | Dtype |
|----|--------------------|-----------------|-------|
| 0  | UsageBand          | 404710 non-null | int8  |
| 1  | fiModelDesc        | 404710 non-null | int16 |
| 2  | fiBaseModel        | 404710 non-null | int16 |
| 3  | fiSecondaryDesc    | 404710 non-null | int16 |
| 4  | fiModelSeries      | 404710 non-null | int16 |
| 5  | fiModelDescriptor  | 404710 non-null | int16 |
| 6  | ProductSize        | 404710 non-null | int8  |
| 7  | fiProductClassDesc | 404710 non-null | int8  |
| 8  | state              | 404710 non-null | int8  |
| 9  | ProductGroup       | 404710 non-null | int8  |
| 10 | ProductGroupDesc   | 404710 non-null | int8  |
| 11 | Drive_System       | 404710 non-null | int8  |
| 12 | Enclosure          | 404710 non-null | int8  |
| 13 | Forks              | 404710 non-null | int8  |
| 14 | Pad_Type           | 404710 non-null | int8  |
| 15 | Ride_Control       | 404710 non-null | int8  |
| 16 | Stick              | 404710 non-null | int8  |
| 17 | Transmission       | 404710 non-null | int8  |
| 18 | Turbocharged       | 404710 non-null | int8  |
| 19 | Blade_Extension    | 404710 non-null | int8  |
| 20 | Blade_Width        | 404710 non-null | int8  |
| 21 | Enclosure_Type     | 404710 non-null | int8  |
| 22 | Engine_Horsepower  | 404710 non-null | int8  |
| 23 | Hydraulics         | 404710 non-null | int8  |
| 24 | Pushblock          | 404710 non-null | int8  |
| 25 | Ripper             | 404710 non-null | int8  |
| 26 | Scarifier          | 404710 non-null | int8  |

# Bulldozers – Preprocesamiento

Dado que el preprocesamiento puede ser un cómputo no despreciable, en este punto podemos guardar nuestro objeto a memoria para poder usarlo desde este punto.

```
save_pickle('./bulldozers-tabular-object.pkl',to)
```

```
to = load_pickle('./bulldozers-tabular-object.pkl')  
xs,y = to.train.xs,to.train.y  
valid_xs,valid_y = to.valid.xs,to.valid.y
```

# Bulldozers – Interpretación

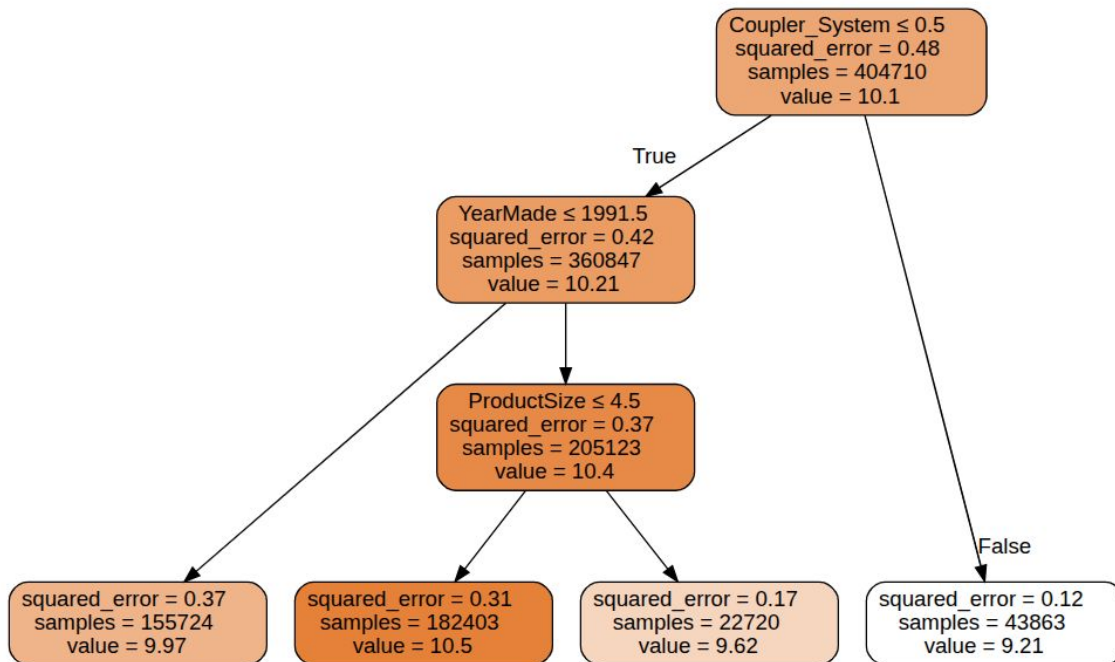
Vamos a comenzar interpretando algo muy simple. Generamos un DecisionTree que no pueda tener más de 4 hojas en total.

Con la funcionalidad *draw\_tree* podemos visualizar este árbol por completo.

```
m = DecisionTreeRegressor(max_leaf_nodes=4)  
m.fit(xs, y);
```

Vamos a visualizar el modelo final

```
draw_tree(m, xs, size=10, leaves_parallel=True, precision=2)
```



# Bulldozers – Decision Tree

El anterior modelo es muy acotado. Vamos entonces a hacer un Decision Tree “completo” y computar nuestra métrica objetivo.

Al hacer esto y computar la métrica sobre el conjunto de entrenamiento nos dió 0 clavado, ¿fin? ¿ganamos?

Ahora creamos un DecisionTree sin profundidad limitada

```
m = DecisionTreeRegressor()  
m.fit(xs, y);
```

Definimos la métrica para el modelo

```
def r_mse(pred,y): return round(math.sqrt(((pred-y)**2).mean()), 6)  
def m_rmse(m, xs, y): return r_mse(m.predict(xs), y)
```

Miramos qué pasa en el conjunto de entrenamiento y en de validación

```
m_rmse(m, xs, y)
```

```
0.0
```

# Bulldozers – Overfitting

Al ver qué pasa en el conjunto de validación vemos que estamos lejos del 0. Nuestro modelo está *overfitteando*.

Si vemos las hojas de nuestro árbol, podemos ver que tenemos casi tantas hojas como filas. Nos estamos excediendo.

Pregunta bonus: si tuviéramos la misma cantidad de hojas que filas ¿a qué modelo se parece esto?

```
m_rmse(m, valid_xs, valid_y)
```

```
0.328661
```

```
m.get_n_leaves(), len(xs)
```

```
(np.int64(324547), 404710)
```

# Bulldozers – Decision Tree

Vamos a hacer un Decision Tree en dónde las hojas agrupen al menos 25 filas.

Ahora sí conseguimos un modelo base en donde no es *tan diferente* nuestra performance en el training set con respecto al validation set.

```
m = DecisionTreeRegressor(min_samples_leaf=25)
m.fit(to.train.xs, to.train.y)
m_rmse(m, xs, y), m_rmse(m, valid_xs, valid_y)
```

```
(0.211717, 0.27031)
```

```
m.get_n_leaves()
```

```
np.int64(12387)
```

# Bulldozer – Random Forest

Una vez conseguido nuestro benchmark, vamos a ir con un Random Forest.

Vemos que ya con un bosque de 40 árboles conseguimos algo considerablemente mejor.

¿Podremos seguir interpretando el modelo?

```
def rf(xs, y, n_estimators=40, max_samples=200_000,  
      max_features=0.5, min_samples_leaf=5, **kwargs):  
    return RandomForestRegressor(n_jobs=-1, n_estimators=n_estimators,  
                                max_samples=max_samples, max_features=max_features,  
                                min_samples_leaf=min_samples_leaf, oob_score=True).fit(xs, y)
```

```
m = rf(xs, y);
```

```
m_rmse(m, xs, y), m_rmse(m, valid_xs, valid_y)
```

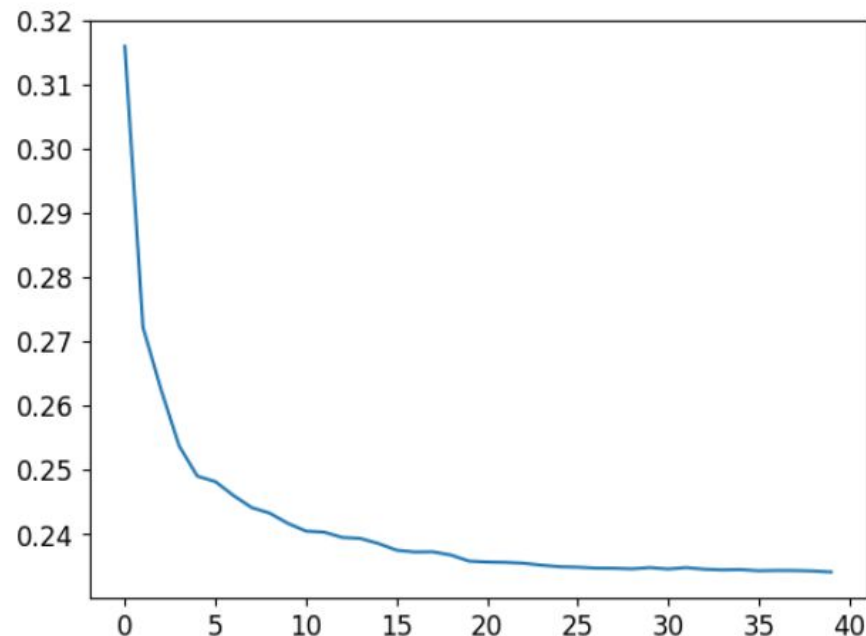
```
(0.170993, 0.234103)
```

# Random Forest – Cantidad de árboles

En primer lugar, podemos ver cómo afecta la cantidad de árboles en el bosque a nuestra métrica en el conjunto de validación.

Esto nos ayuda a entender cómo mejora el modelo pero no qué está haciendo.

```
preds = np.stack([t.predict(valid_xs) for t in m.estimators_])  
plt.plot([r_mse(preds[:i+1].mean(0), valid_y) for i in range(40)]);
```



# Random Forest – Feature Importance

El modelo Random Forest de Scikit-learn ya trae incorporado un atributo *feature\_importances\_* que nos da un valor numérico de qué tan importante es una columna.

Ahora podemos empezar a visualizar qué está teniendo en cuenta el modelo para hacer las predicciones.

```
def rf_feat_importance(m, df):  
    return pd.DataFrame({'cols':df.columns, 'imp':m.feature_importances_  
                        }).sort_values('imp', ascending=False)
```

```
fi = rf_feat_importance(m, xs)  
fi[:10]
```

|    | cols               | imp      |
|----|--------------------|----------|
| 58 | YearMade           | 0.186833 |
| 6  | ProductSize        | 0.127272 |
| 30 | Coupler_System     | 0.075358 |
| 7  | fiProductClassDesc | 0.070345 |
| 32 | Hydraulics_Flow    | 0.059941 |
| 66 | saleElapsed        | 0.049191 |
| 55 | ModelID            | 0.048763 |
| 3  | fiSecondaryDesc    | 0.045518 |
| 31 | Grouser_Tracks     | 0.037843 |
| 12 | Enclosure          | 0.036103 |

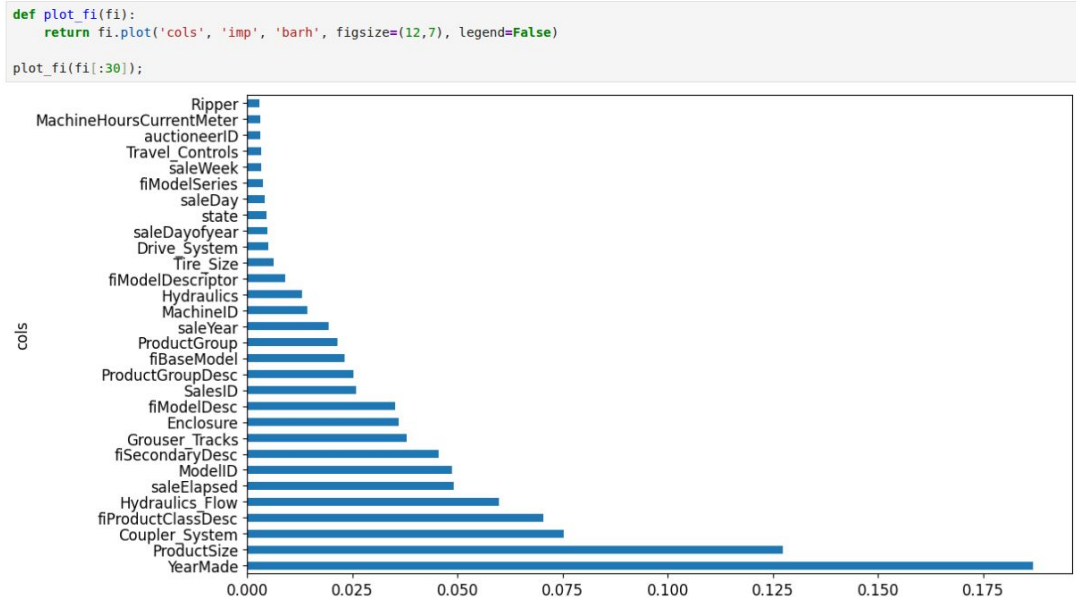
# Random Forest – Cálculo de feature importance

*The way these importances are calculated is quite simple yet elegant. The feature importance algorithm loops through each tree, and then recursively explores each branch. At each branch, it looks to see what feature was used for that split, and how much the model improves as a result of that split. The improvement (weighted by the number of rows in that group) is added to the importance score for that feature. This is summed across all branches of all trees, and finally the scores are normalized such that they add to 1.*

Otra manera de calcular la feature importance: una vez entrenado nuestro modelo, tomamos la columna a la cual queremos calcularle su importancia, la shuffleamos y volvemos a utilizar el modelo de predicción. El deterioro en la medida de predicción se puede utilizar como medida de importancia del feature.

# Random Forest – Feature Importance

Ahora tenemos un gráfico de *feature importance* que no tiene *mucho* que envidiarle en cuanto a interpretabilidad a un modelo lineal.



# Random Forest – Feature Selection

Esta información se puede utilizar para hacer Feature Selection.

Al hacerlo no solo conseguimos un modelo más simple sino que además está funcionando mejor sobre el conjunto de validación.

```
to_keep = fi[fi.imp>0.005].cols  
len(to_keep)
```

```
21
```

```
xs_imp = xs[to_keep]  
valid_xs_imp = valid_xs[to_keep]
```

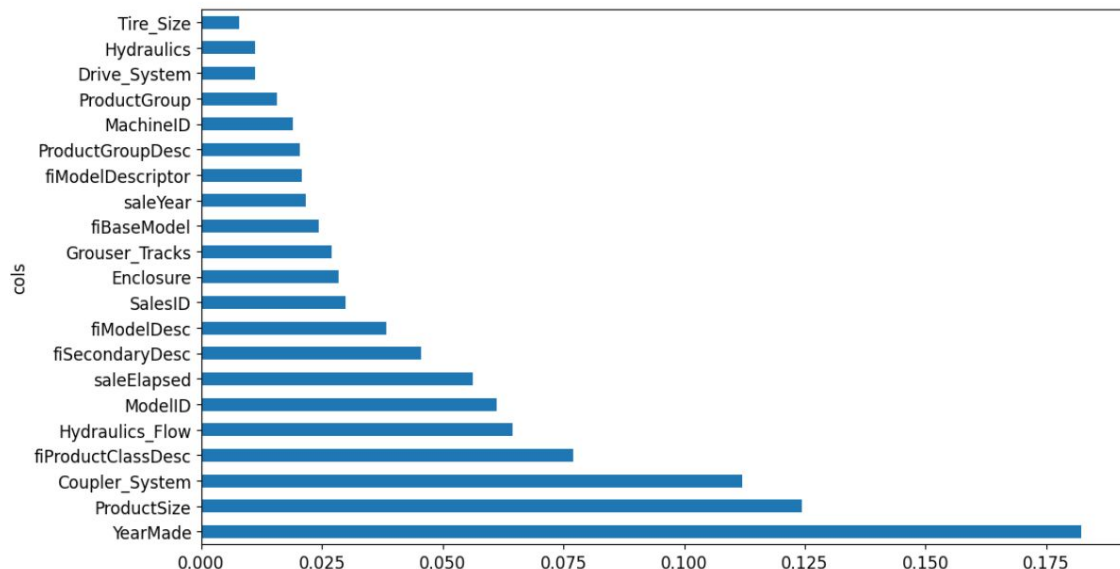
```
m = rf(xs_imp, y)
```

```
m_rmse(m, xs_imp, y), m_rmse(m, valid_xs_imp, valid_y)  
(0.181015, 0.231821)
```

# Random Forest – Feature Selection

Ahora con muchas menos columnas podemos tener un gráfico más limpio y claro.

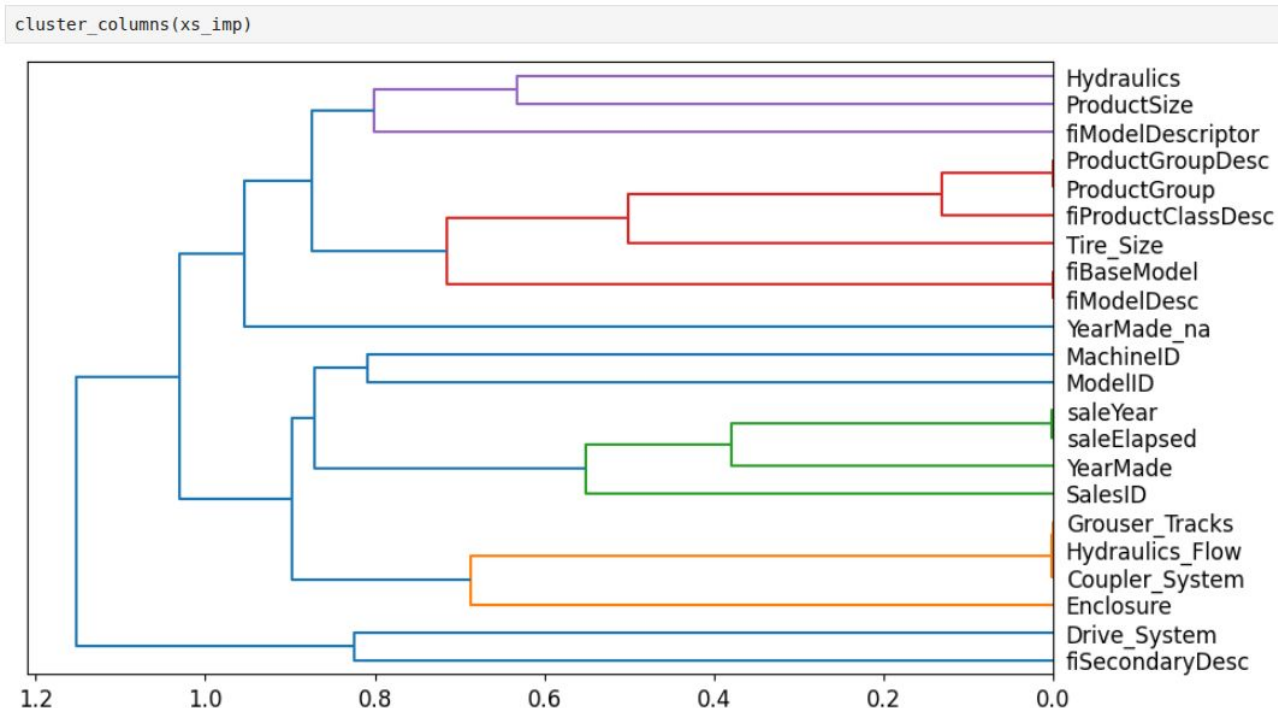
```
plot_fi(rf_feat_importance(m, xs_imp));
```



# Random Forest – Interpretación

Otra forma posible de interpretación (y de Feature Selection) es entender qué columnas son *parecidas* para el modelo.

Para esto podemos hacer un clustering jerárquico aglomerativo (¿un qué?) que vaya agrupando las columnas.



# Random Forest – Cluster Columns

En general, cuando clusterizamos, lo hacemos en función de una medida de distancia de puntos en el espacio.

Este caso es parecido pero en vez de tomar puntos y una distancia euclídea en el espacio, los “puntos” a clusterizar serán las columnas o features del dataset y en vez de distancia euclídea se utiliza la medida de rank correlation.

El rank correlation entre dos columnas lo que mide es la correlación que tienen los puntos en cada columna según su orden dentro de esa columna, sin importar el valor absoluto (dado que a los árboles de decisión el valor absoluto no es algo que les importe).

# Random Forest – Feature Selection

El clustering sirve como interpretación pero también como material para hacer Feature Selection. Podemos ver qué sucede al sacar un representante de cada cluster y así decidir qué features descartar.

```
def get_oob(df):  
    m = RandomForestRegressor(n_estimators=40, min_samples_leaf=15,  
                             max_samples=50000, max_features=0.5, n_jobs=-1, oob_score=True)  
    m.fit(df, y)  
    return m.oob_score_
```

```
get_oob(xs_imp)
```

```
0.8778402552203507
```

```
{c:get_oob(xs_imp.drop(c, axis=1)) for c in (  
    'saleYear', 'saleElapsed', 'ProductGroupDesc', 'ProductGroup',  
    'fiModelDesc', 'fiBaseModel',  
    'Hydraulics_Flow', 'Grouser_Tracks', 'Coupler_System')}
```

```
{'saleYear': 0.8775896228512217,  
 'saleElapsed': 0.8732593254588217,  
 'ProductGroupDesc': 0.878550063354707,  
 'ProductGroup': 0.8781817887980742,  
 'fiModelDesc': 0.8765091824171777,  
 'fiBaseModel': 0.8768243055074915,  
 'Hydraulics_Flow': 0.8779969217835307,  
 'Grouser_Tracks': 0.878239377304804,  
 'Coupler_System': 0.8779898290795644}
```

# Random Forest – Feature Selection

Elegimos el mejor representante de cada cluster para descartar.

Ahora obtenemos un modelo aún más simple y que no pierde poder de generalización.

```
to_drop = ['saleYear', 'ProductGroupDesc', 'fiBaseModel', 'Grouser_Tracks']  
get_oob(xs_imp.drop(to_drop, axis=1))
```

```
0.8755805712569211
```

```
xs_final = xs_imp.drop(to_drop, axis=1)  
valid_xs_final = valid_xs_imp.drop(to_drop, axis=1)
```

```
save_pickle('./xs_final.pkl', xs_final)  
save_pickle('./valid_xs_final.pkl', valid_xs_final)
```

```
xs_final = load_pickle('./xs_final.pkl')  
valid_xs_final = load_pickle('./valid_xs_final.pkl')
```

```
m = rf(xs_final, y)  
m_rmse(m, xs_final, y), m_rmse(m, valid_xs_final, valid_y)
```

```
(0.183009, 0.232705)
```

# Random Forest – Partial Dependence

Otro análisis de interpretación del modelo que se puede realizar es el de Partial Dependence.

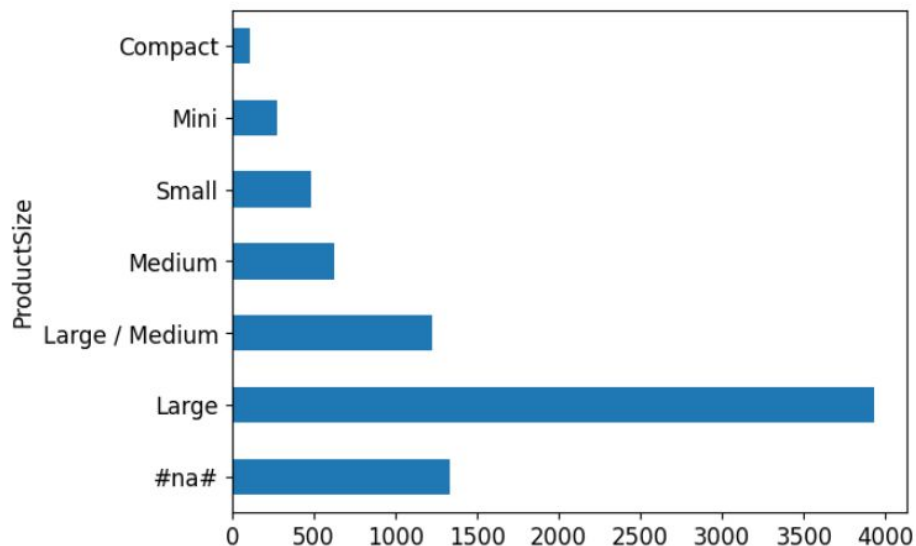
La idea en este análisis es observar cómo depende el valor a predecir de una columna en particular.

Para hacer esto podríamos tomar la columna a analizar y por cada valor de la columna simplemente tomar el promedio de la variable target. Esto no es incorrecto pero podemos hacer algo un poco mejor.

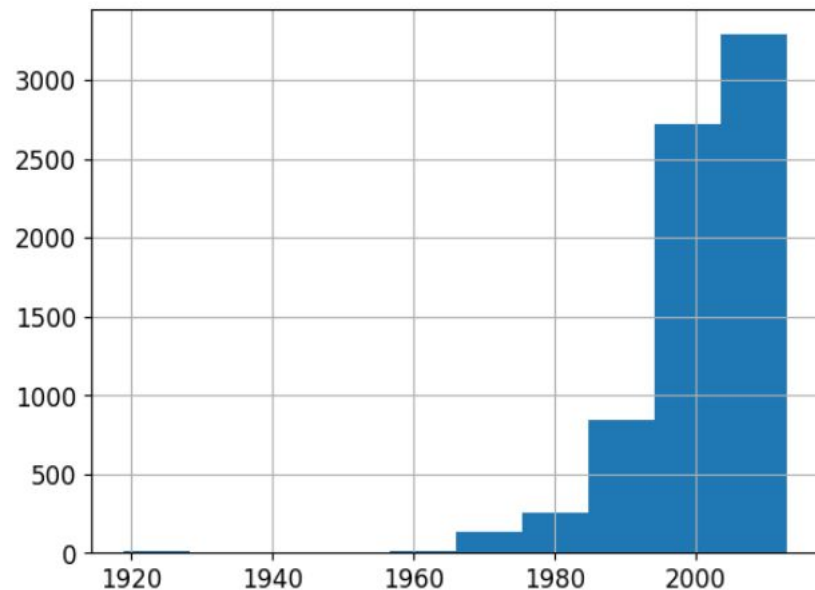
Vamos a hacerlo con las columnas *ProductSize* y *YearMade* que fueron las dos de mayor feature importance.

# Random Forest – Partial Dependence

```
p = valid_xs_final['ProductSize'].value_counts(sort=False).plot.barh()  
c = to.classes['ProductSize']  
plt.yticks(range(len(c)), c);
```



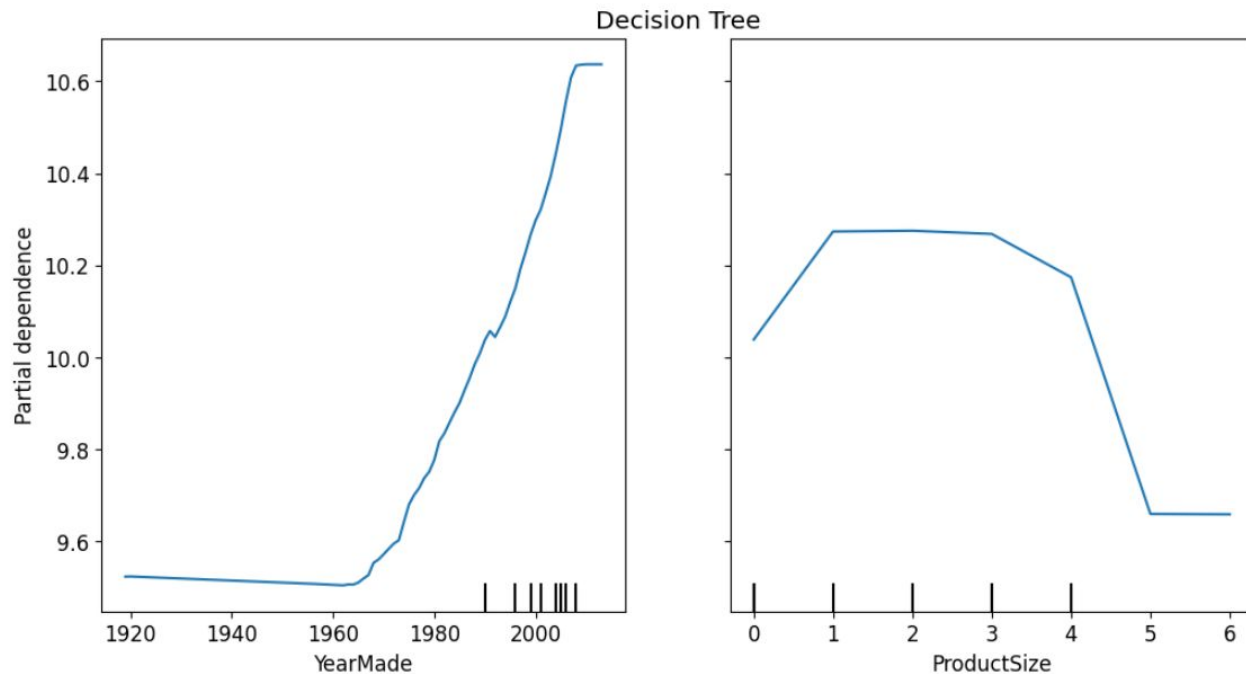
```
ax = valid_xs_final['YearMade'].hist()
```



# Random Forest – Partial Dependence

```
from sklearn.inspection import PartialDependenceDisplay

fig, ax = plt.subplots(figsize=(12, 6))
ax.set_title("Decision Tree")
tree_disp = PartialDependenceDisplay.from_estimator(m, valid_xs_final, ['YearMade', 'ProductSize'], ax=ax)
```



# Random Forest – Partial Dependence

Dijimos que no nos conformábamos con tomar el promedio de la variable target para cada valor del feature a analizar. Entonces, ¿cómo se calcula esta *partial dependence*?

Lo que hacemos es, una vez entrenado nuestro modelo, reemplazar la columna a analizar por cada uno de sus valores posibles y calcular el promedio de las predicciones. Es decir, por cada valor que puede tomar la columna, reemplazamos toda esa columna por dicho valor en cada fila, utilizamos el modelo para predecir cada nueva fila y luego tomamos el promedio de la predicción.

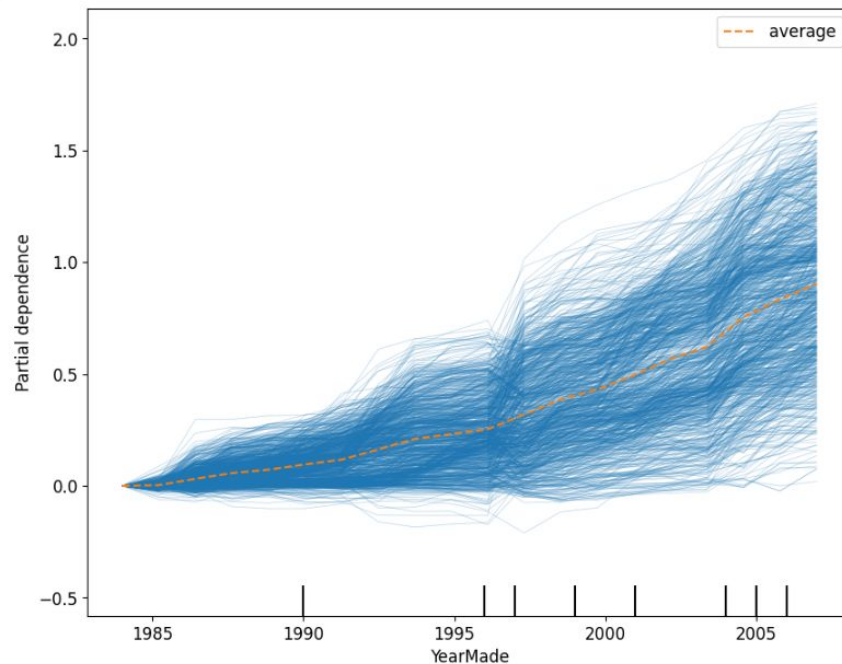
# Random Forest – ICE plot

El Partial Dependence Plot toma el promedio para cada valor posible de la columna a analizar.

El Individual Conditional Expectation plot (ICE) grafica la evolución de todas las filas de nuestro dataset.

```
fig, ax = plt.subplots(figsize=(10, 8))

common_params = {"grid_resolution": 20, "random_state": 0}
features_info = {"features": ['YearMade'], "kind": "both", "centered": True}
display = PartialDependenceDisplay.from_estimator(
    m,
    valid_xs_final,
    **features_info,
    ax=ax,
    **common_params,
)
```



# Random Forest- Contributions

Otro análisis posible es el de contribuciones. En este análisis podemos ver para cada fila como se llega al valor final predicho.

Para esto se toma el valor promedio de la columna a predecir y luego se van mostrando las contribuciones que tiene cada columna para llegar al valor final de la fila.

```
from treeinterpreter import treeinterpreter
from waterfall_chart import plot as waterfall
```

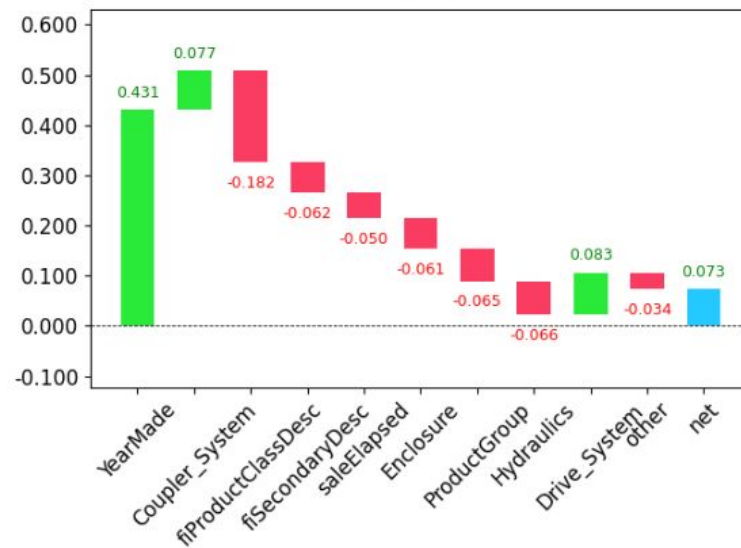
```
row = valid_xs_final.iloc[:5]
```

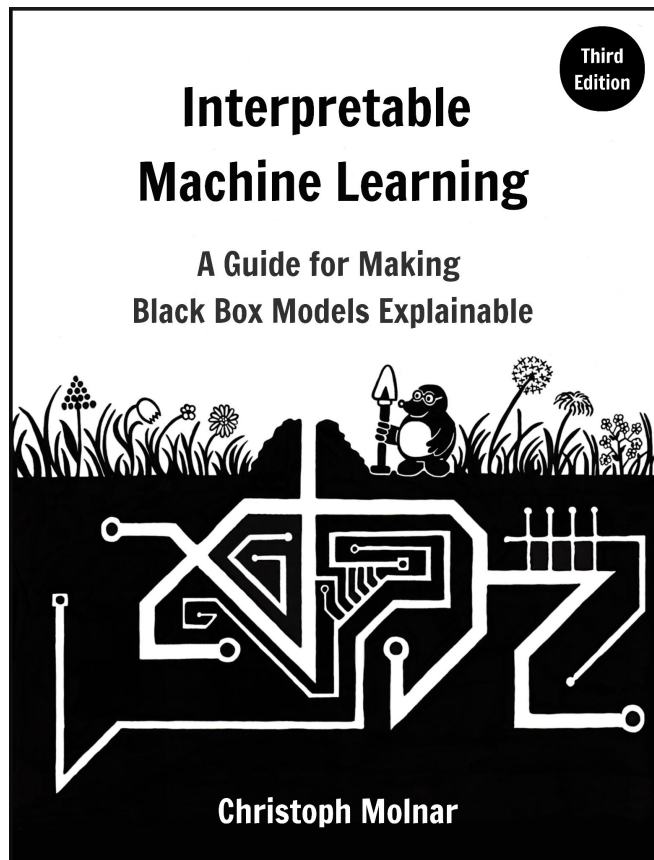
```
prediction,bias,contributions = treeinterpreter.predict(m, row.values)
```

```
prediction[4], bias[4], contributions[4].sum()
```

```
(array([10.17705413]), 10.10434023018098, 0.07271390343623342)
```

```
waterfall(valid_xs_final.columns, contributions[4], threshold=0.08,
          rotation_value=45,formatting='{:,.3f}');
```





# Random Forest – Resumen

Es un ensamble muy poderoso para usar en la práctica en problemas tabulares o estructurados.

Son *bastante inmunes* al overfitting.

No necesitan un gran preprocesamiento. No necesitan escalar features o regularizaciones particulares.

No todo es felicidad: no tienen ninguna capacidad de extrapolación.

# Random Forest – Extrapolación

Los árboles de decisión siempre están tomando promedios de muestras que ven en el conjunto de entrenamiento.

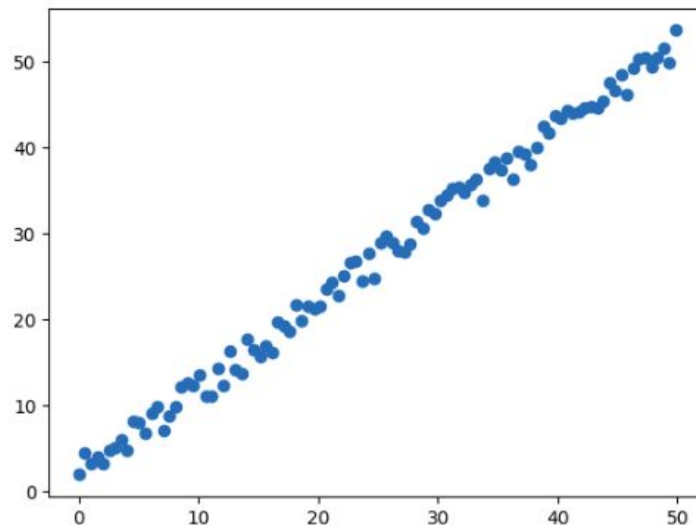
Si una nueva muestra está muy lejos de lo que vieron, no tienen nada para hacer al respecto.

Veamos un ejemplo concreto con una recta creciente.

¿Qué va a predecir un random forest con valores de  $x$  mayores a 50?

```
import numpy as np
import random
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
```

```
x = np.linspace(0,75,150)
x_train = x[:100]
y = np.linspace(0,50,100)
y = x_train + 4*np.random.rand(100)
plt.scatter(x_train, y);
```



# Random Forest – Extrapolación

Un RandomForest no puede hacer ningún tipo de extrapolación.

En este caso caerá por las hojas del árbol que lo lleven más a la derecha del eje x y replicará todas las veces ese valor.

De todas maneras, hay formas de tratar evitar estos problemas.

```
rf = RandomForestRegressor()  
rf.fit(x_train.reshape(-1,1), y)  
predictions = rf.predict(x.reshape(-1,1))  
plt.scatter(x_train, y);  
plt.scatter(x, predictions);
```

